

Scalability Analysis of Serial, Shared-Memory, Distributed-Memory, and Hybrid Parallel Heat Transfer Simulation on a High-Performance Computing System

Roni Butbul
Department of Computing Sciences
Coastal Carolina University
Conway, SC, USA

Shawn McDonald
Department of Computing Sciences
Coastal Carolina University
Conway, SC, USA

Thomas Cahill
Department of Computing Sciences
Coastal Carolina University
Conway, SC, USA

Raymond Pridgen
Department of Computing Sciences
Coastal Carolina University
Conway, SC, USA

Abstract—In this paper we present a systematic performance and scalability analysis of four parallel programming paradigms applied to a two-dimensional heat diffusion simulation using a nine-point stencil on grids ranging from $5,000 \times 5,000$ to $40,000 \times 40,000$ elements. Implementations in POSIX threads (Pthreads), OpenMP, MPI, and a hybrid MPI+OpenMP model are evaluated across thread and process counts of $p \in \{1, 2, 4, 8, 16\}$ on the Expanse high-performance computing cluster, yielding 100 total experiments. Performance was evaluated through wall-clock time, isolated computation time, and derived communication and synchronization overhead. Strong scaling analysis reveals that MPI achieves the highest overall speedup of $5.97\times$ at $p = 16$ on the largest grid, while also exhibiting superlinear speedup of $2.18\times$ at $p = 2$ —attributed most likely to a cache working-set reduction effect as per-rank subgrid data fits within available cache capacity. OpenMP demonstrates the most favorable weak scaling behavior and the lowest synchronization overhead but it suffers from a consistent 17–30% single-thread penalty relative to the serial baseline due to runtime costs. The Hybrid implementation, despite combining both distributed- and shared-memory parallelism, fails to surpass MPI in all configurations, with communication overhead reaching 79.5% of total runtime at $p = 16$ for $n = 20,000$. Amdahl’s law analysis shows that the stencil computation kernel itself approaches near-perfect parallelism. MPI achieves $17.1\times$ speedup on isolated computation time at $p = 16$. This confirms that all observed performance ceilings arise from communication and synchronization overhead rather than inherent good programming and easily parallelization stencils. These results demonstrate that optimal parallel strategy is both problem-size- and process-count-dependent, and that increasing p does not monotonically improve performance in three of the four implementations studied.

I. INTRODUCTION

Modern processors and data centers generate enormous amounts of heat. As chip designs become denser and AI workloads start to push hardware closer to its thermal limits, the ability to simulate how heat spreads across a surface has become increasingly important. Thermal simulation helps

engineers predict hotspots, design better cooling systems, and avoid hardware failures before they happen.

The core computation behind these simulations is basically a grid update problem. Each cell in a 2D grid stores a temperature value, and at every time step that value is updated based on its neighbors. The math is simple, but the scale is not. A $40,000 \times 40,000$ grid contains 1.6 billion cells. Doing that serially, iteration after iteration would be computationally expensive. Parallelism is the obvious answer, but adding more threads or processes does not automatically make things faster. Synchronization, communication, and memory overhead all cut into the gains in some way shape or form.

This project implements the same nine-point stencil heat diffusion simulation five ways—serial, Pthreads, OpenMP, MPI, and hybrid MPI+OpenMP—and benchmarks all of them across grid sizes from $5,000 \times 5,000$ to $40,000 \times 40,000$ with $p \in \{1, 2, 4, 8, 16\}$ threads or processes on the Expanse HPC cluster.

The central questions driving this analysis are:

- How much faster are the parallel implementations compared to serial?
- Which implementation scales best as problem size and process count grow?
- How does overhead change as p increases, and when does it dominate?
- Is there an optimal p for each paradigm?
- Does the hybrid implementation actually outperform pure MPI?

The results tell us that MPI is the fastest overall, achieving a peak speedup of $5.97\times$ at $p = 16$ and even superlinear speedup at $p = 2$ due to cache effects. Pthreads and OpenMP provide solid speedup at moderate thread counts but struggle at higher numbers. The hybrid implementation never consistently beats

pure MPI, and in several configurations its overhead makes it one of the slowest options despite using the most resources.

II. BACKGROUND

To quickly touch on heat transferring we bring up heat diffusion. Heat diffusion describes how thermal energy spreads through a material over time. In a two-dimensional medium, temperature at any point changes based on the temperatures of surrounding points. Basically, hotter areas transfer energy to cooler ones until the system reaches equilibrium. Computationally, this process is modeled by partitioning the domain into a grid and repeatedly updating each cell based on its neighbors, which is exactly what a stencil computation does.

A. The Nine-Point Stencil

A stencil computation updates each cell in a grid using a fixed pattern of neighboring values. This project uses a nine-point stencil, which averages a cell with all eight of its neighbors:

$$new[i, j] = \frac{1}{9} \sum_{di=-1}^1 \sum_{dj=-1}^1 old[i + di, j + dj]$$

What this is on a higher level is an approximation of the discrete Laplacian edge over the local neighborhood, including diagonal neighbors so heat spreads in all directions, not just up, down, left, and right. An important detail is that floating-point addition is not associative so the order the nine values are added affects the final result. All parallel implementations must use the same addition order as the serial version to produce matching output.

Boundary cells are fixed throughout the simulation. The left and right edges are held at 1.0 to represent a constant heat source, and the top and bottom edges are held at 0.0 to represent a constant cooling source. Only interior cells are updated each iteration.

B. Computational Complexity

For a grid of size $N \times N$ running for t iterations, the total work is:

$$W = O(N^2 \cdot t)$$

Because the grid is two-dimensional, work grows quadratically with N . Doubling N does not double the work — it quadruples it. Going from $N = 10,000$ to $N = 40,000$ increases the number of cells by a factor of 16. At $N = 40,000$ the grid has 1.6 billion cells, which motivates the need for parallelism.

C. Parallel Computing Models

This project uses three different parallel programming models, each with different assumptions about memory and communication.

In a shared-memory model, all threads live in the same process and can directly access the same grid. There is no messaging required between threads, but threads still need to

synchronize at the end of each iteration before starting the next one. Pthreads and OpenMP both use this model. Pthreads gives the programmer full manual control over threads and barriers. OpenMP handles most of that automatically through compiler directives, which makes it easier to write but harder to reason about in terms of performance.

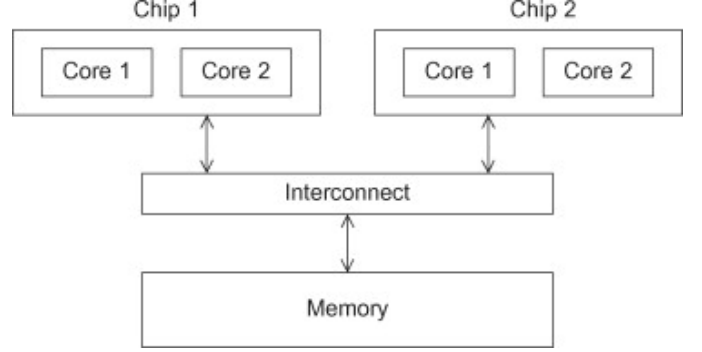


Fig. 1. Shared-memory model: all threads access a common grid.

MPI uses a distributed-memory model where each process has its own private memory and must explicitly send data to other processes. The grid is split into horizontal bands, and each rank owns a chunk of rows. At each iteration, neighboring ranks need to exchange their boundary rows. These are called ghost cells before they can compute their local updates. This exchange is where most of MPI's overhead comes from.

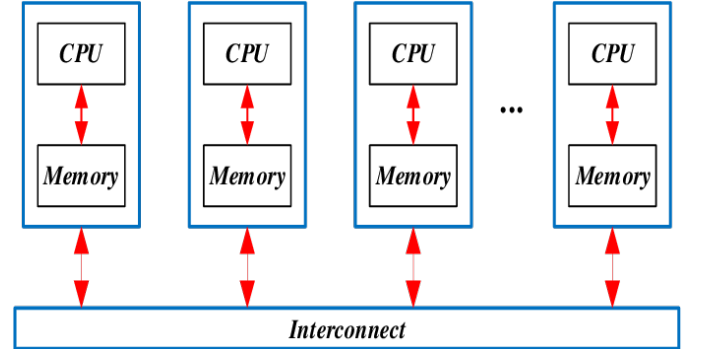


Fig. 2. Distributed-memory model: each rank owns private rows and exchanges ghost cells with neighbors.

As illustrated in Figures 1 and 2, the key structural difference between the two models is whether the grid lives in one shared address space or is partitioned across private memory regions.

The hybrid approach runs MPI across nodes and OpenMP within each node. The idea is to get the best of both worlds. In practice it can also mean getting the overhead of both, and whether it actually helps depends on the problem size and how the work divides across the hardware.

D. Speedup, Efficiency, and Amdahl's Law

Speedup measures how much faster a parallel run is compared to serial:

$$S(p) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(p)}$$

Efficiency measures how well the processing elements are actually being used:

$$E(p) = \frac{S(p)}{p}$$

Perfect scaling means $S(p) = p$ and $E(p) = 1.0$, which never happens in practice. Amdahl's Law gives the theoretical speedup limit for a program where fraction f of the work can be parallelized:

$$S(p) \leq \frac{1}{(1-f) + \frac{f}{p}}$$

As p grows, the speedup ceiling approaches $\frac{1}{1-f}$, meaning even a small serial portion puts a hard cap on how fast the program can get. For this stencil, that serial portion includes file I/O, grid initialization, and sequential communication. On top of Amdahl's ceiling, real performance is further reduced by synchronization cost, memory bandwidth, and communication latency, all of which show up clearly in the results.

III. DESIGN AND IMPLEMENTATION

A. Overall Program Structure

The project was implemented in C and organized into several separate programs. The overall workflow follows a generate, compute, verify, and analyze structure.

First, an input matrix is generated using a utility program. The matrix is written to a binary file. The file begins with two integers representing the number of rows and columns, followed by the grid values stored as double-precision values.

Next, each stencil implementation reads the same input file and performs the heat transfer simulation for a fixed number of iterations. After the simulation completes, each implementation writes its final grid to an output file using the same binary format.

Finally, a verification program compares output files from different implementations. This ensures that the parallel versions produce the same final result as the serial baseline.

B. Serial Implementation

The serial implementation serves as the baseline for correctness and performance. It reads the full matrix into memory, allocates a second matrix for updated values, and then repeatedly applies the stencil update.

For each iteration, the program first preserves the boundary values. Then it updates every interior cell using the 9-point average. After all cells are updated, the program swaps the current and next buffers. This avoids copying the entire matrix after every iteration.

The serial implementation is straightforward and easy to reason about. Because it uses only one processing element, it avoids synchronization and communication overhead. However, it cannot take advantage of multiple cores, so it becomes very slow for large problem sizes.

In this project, the serial version also records timing information. It prints a metadata line identifying the version, number of threads, number of ranks, matrix dimensions, and iteration count. It then prints overall runtime, computation time, and overhead time. This consistent output format makes it easier to parse results into a CSV file for analysis. The serial program uses this standard timing format in its final output.

C. Pthreads Implementation

The pthread implementation introduces shared-memory parallelism. Instead of one thread processing the entire grid, the interior rows are divided among multiple pthreads. Each thread receives a start row and end row and computes only that portion of the grid.

The main advantage of this approach is direct control. The programmer explicitly creates the threads, assigns work, and synchronizes them. In this implementation, rows are divided as evenly as possible. If the number of rows does not divide evenly by the number of threads, the extra rows are distributed among the first few threads.

Synchronization is necessary because the stencil computation uses double buffering. All threads must finish writing their assigned portion of the next matrix before any thread swaps the current and next pointers. To handle this, the implementation uses a custom barrier. The barrier is used once after computation and once after the buffer swap.

Only one thread handles boundary copying, debug printing, and pointer swapping. This reduces the risk of multiple threads modifying shared control variables at the same time. After the swap, all threads wait at a second barrier before beginning the next iteration.

This design is effective because the interior cells can be updated independently during each iteration. Since each thread writes to a separate set of rows, there is no need for locks during the main computation. The main overhead comes from thread creation, barrier synchronization, and memory contention.

D. OpenMP Implementation

The OpenMP implementation also uses shared-memory parallelism, but it does so using compiler directives instead of explicit thread creation. This makes the OpenMP code shorter and easier to read than the pthread version.

The OpenMP program uses a parallel region inside each iteration. Boundary copying is parallelized using OpenMP for loops, and the interior stencil update uses a collapsed nested loop. The collapse directive allows OpenMP to distribute the two-dimensional loop space across threads more evenly.

Static scheduling is used so that loop iterations are distributed predictably across threads. This is appropriate for this problem because each cell update performs roughly the

same amount of work. There is no major imbalance between different regions of the matrix.

The main benefit of OpenMP is simplicity. The runtime handles thread management and scheduling. However, that also means the programmer has less direct control over thread behavior. This can introduce overhead or variability, especially at higher thread counts.

In the experimental results, OpenMP improves significantly over the serial version, but it is not always as fast as pthreads or MPI. This is likely due to runtime overhead, scheduling costs, and shared-memory contention.

E. MPI Implementation

The MPI implementation uses distributed-memory parallelism. Instead of creating threads inside one process, the program launches multiple MPI processes, called ranks. Each rank owns a section of the grid.

The grid is divided by rows. Each MPI rank receives a contiguous band of rows, along with extra halo rows needed for communication. Before each stencil iteration, ranks exchange boundary information with neighboring ranks. A rank must receive the row above and the row below its local region so that it can correctly compute stencil values near partition boundaries.

This halo exchange is one of the most important parts of the MPI implementation. Without it, each rank would not have the necessary neighbor values for cells near the top or bottom of its local subdomain.

After the computation finishes, the MPI implementation gathers the distributed matrix back into one output file. This allows the MPI result to be verified against the serial result using the same verification tool. The MPI implementation also uses barriers around the timed computation region so that timing reflects the coordinated execution of all ranks.

The MPI approach introduces communication overhead, but it reduces shared-memory contention. This is one reason MPI performs especially well for the largest problem sizes. When the grid becomes very large, the benefit of splitting work across processes outweighs the communication cost.

F. Hybrid MPI+OpenMP Implementation

The hybrid implementation combines MPI and OpenMP. MPI divides the grid across ranks, and OpenMP threads compute the rows inside each rank. In this project, the hybrid configuration was mapped so that the total parallel value P was approximated using a combination of ranks and threads.

For example, at $P = 16$, the hybrid implementation used 4 MPI ranks and 4 OpenMP threads per rank. This creates 16 total workers, but the structure is different from pure MPI or pure OpenMP.

The hybrid implementation uses MPI for reading, scattering, halo exchange, and gathering output. Inside each rank, OpenMP parallelizes the local stencil computation. The implementation also initializes MPI using `MPI_Init_thread` with `MPI_THREAD_FUNNELED`, which ensures that MPI

calls are made safely while OpenMP threads handle computation.

The hybrid approach is useful because many real HPC systems are organized as clusters of multi-core nodes. MPI can distribute work across nodes, while OpenMP can use multiple cores within a node. However, in this experiment, the hybrid version did not consistently outperform pure MPI. This suggests that the added thread-level overhead did not provide enough benefit for this particular implementation and resource configuration.

G. Automation and CSV Logging

The experiments were automated using SLURM batch scripts. The scripts compiled the code, generated input files, executed each implementation, and collected timing output into CSV files.

A mini test script was used before the full experiment to confirm correctness and ensure that the output format was parseable. The mini script ran all implementations on a 5000 by 5000 matrix and verified that pthreads, OpenMP, MPI, and hybrid all produced the same output as serial. It also wrote timing data to a CSV file.

This automation was important because the final experiment involved many runs. Manually running each configuration would increase the chance of mistakes. Using a script ensured that all versions were run with consistent arguments and that the data was collected in a repeatable way.

IV. EXPERIMENTAL EVALUATION

A. Setup

All of our experiments were conducted on the shared partition of the Expanse high-performance computing cluster at the San Diego Supercomputer Center. Expanse compute nodes are equipped with two 64-core AMD EPYC 7742 processors running at 2.25 GHz and 256 GB of DDR4 memory. The system runs Rocky Linux and uses the SLURM workload manager for job scheduling [5]. The simulation was compiled using `gcc` with `-O2` optimization and `OpenMPI` for MPI builds. All five implementations were evaluated across grid sizes $N \in \{5000, 10000, 20000, 40000\}$ and thread or process counts $p \in \{1, 2, 4, 8, 16\}$ at a fixed $t = 13$ iterations, yielding 100 total experiments. The value $t = 13$ was selected because it caused the largest serial configuration ($N = 40,000$) to run for approximately four minutes, satisfying the assignment requirement while keeping total cluster usage manageable.

For each run, three values were recorded:

- T_{overall} : total wall-clock time from program start to finish, including I/O, setup, computation, communication, and output.
- $T_{\text{computation}}$: time spent exclusively inside the main stencil iteration loop.
- $T_{\text{other}} = T_{\text{overall}} - T_{\text{computation}}$: derived overhead time capturing everything outside the stencil loop, including file I/O, memory allocation, communication, and synchronization.

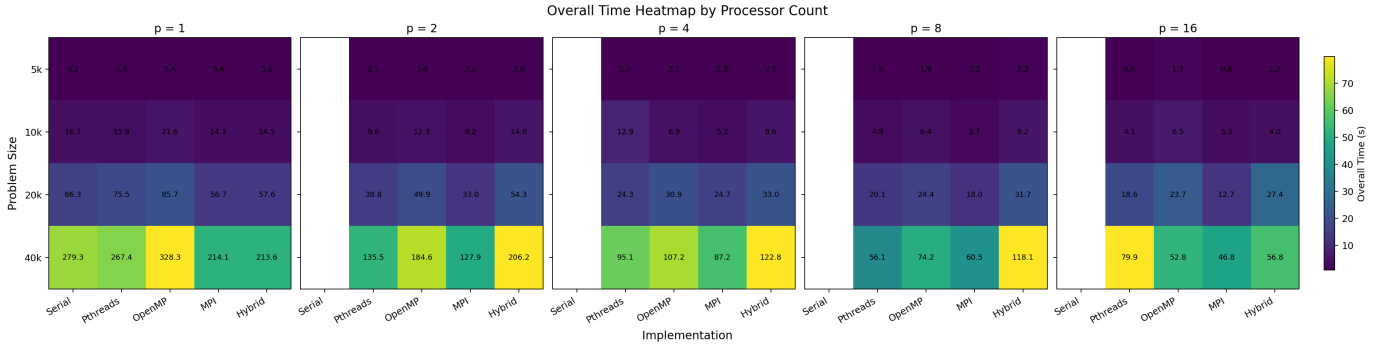


Fig. 3. Heatmap of total wall-clock time across all implementations, grid sizes, and process counts. Darker cells indicate longer runtimes.

The serial implementation at each grid size serves as the baseline for all speedup and efficiency calculations. Because the experiment ran on a shared partition, some timing noise was expected. Isolated anomalies, in particular a spike in Pthreads overhead at $N = 10,000$, $p = 4$ are consistent with system-level interference rather than algorithmic problems. All parallel implementations were verified to produce output identical to the serial baseline before performance data was collected.

B. Overall Runtime

Figure 3 provides a bird’s-eye view of all 100 experiments. The quadratic growth of the stencil workload is immediately visible. The serial baseline grows from 4.2 seconds at $N = 5,000$ to 279.3 seconds at $N = 40,000$. This is a $67\times$ increase for an $8\times$ increase in N , consistent with $O(N^2 \cdot t)$ complexity. Doubling N does not double the work; it quadruples it, which is why the bottom rows of the heatmap are so much darker than the top.

All parallel implementations reduce runtime substantially. At $N = 40,000$ and $p = 16$, the fastest implementation (MPI) finishes in 46.8 seconds versus 279.3 seconds for serial — a $5.97\times$ improvement. One result stands out immediately even at this level: OpenMP at $p = 1$ is visibly slower than the serial baseline across all grid sizes, running 17–30% slower depending on N . This is caused by OpenMP runtime initialization overhead that is present even with a single thread.

C. Speedup Analysis

Figure 4 shows overall speedup $S(p) = T_{\text{serial}}/T_{\text{overall}}(p)$ for all four parallel implementations. The dashed ideal line represents perfect linear scaling where doubling p doubles speedup.

MPI is the strongest performer overall. It reaches $5.97\times$ at $N = 40,000$, $p = 16$ and — notably — achieves superlinear speedup at $p = 2$ across every grid size, peaking at $2.18\times$ for the largest grid. This is not a measurement error. When a single MPI process owns the full $40,000 \times 40,000$ grid, the working set far exceeds available cache. Splitting across two ranks halves the per-rank data, allowing it to fit more effectively in cache and reducing memory access latency per iteration. This cache working-set effect produces speedup beyond the linear

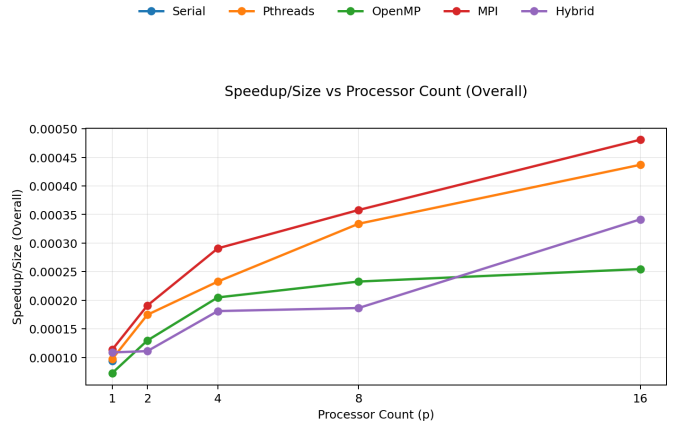


Fig. 4. Overall speedup vs. p for all implementations and grid sizes. Dashed line indicates ideal linear speedup.

ideal and is one of the most significant findings of this study. The same effect is notable at all four matrix sizes, confirming it is a structural property of the MPI decomposition rather than noise.

Pthreads scales well through $p = 8$ at $N = 40,000$, reaching $4.97\times$, but then regresses to $3.49\times$ at $p = 16$. This is a clear reversal rather than a gradual slowdown, and it makes Pthreads the only implementation where the best performance is not at the maximum p . At $p = 16$, isolated computation time is 55.8 seconds against an ideal of 15.0 seconds. This is a $3.7\times$ gap and suggests severe cache thrashing or maybe barrier contention at that partition size. A related anomaly occurs at $N = 10,000$, $p = 4$, where overall runtime is actually worse than at $p = 2$. This isolated spike is most likely a false sharing event, where thread boundary rows happen to share cache lines at that specific partition boundary, causing repeated cache invalidations on every stencil write.

OpenMP improves steadily and reaches $5.29\times$ at $N = 40,000$, $p = 16$, making it the second-fastest implementation at maximum parallelism interestingly. However, it plateaus noticeably between $p = 8$ and $p = 16$ at smaller grid sizes, and its $p = 1$ baseline is consistently worse than serial, which artificially inflates its apparent speedup at higher thread counts

relative to what a fair single-threaded comparison would show.

Hybrid shows the weakest and most uneven scaling curve contrary to our original belief. Going from $p = 1$ to $p = 2$ has no benefit at any grid size. At $N = 40,000$, runtime barely changes from 213.6 seconds to 206.2 seconds. The curve then flattens significantly between $p = 4$ and $p = 8$ before recovering at $p = 16$. This step-wise pattern points to the irregular interaction between MPI rank boundaries and OpenMP thread scheduling within each rank. When p doubles but the rank-to-thread ratio stays the same, the added parallelism doesn't translate cleanly.

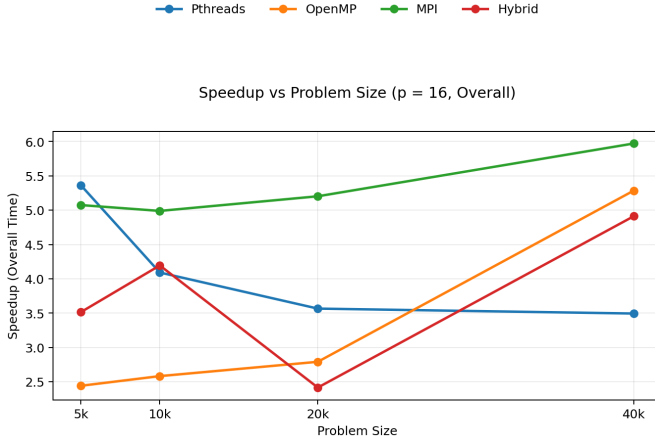


Fig. 5. Overall speedup vs. N at $p = 16$ for all implementations. Speedup generally improves with problem size, consistent with Amdahl's Law.

Figure 5 fixes $p = 16$ and shows how speedup changes as N grows. For most implementations, speedup increases with problem size. This is consistent with Amdahl's Law. As the problem grows, the parallel fraction of the work dominates more strongly over fixed serial costs, so more of the runtime actually benefits from parallelism. MPI shows the clearest improvement, growing from $5.07\times$ at $N = 5,000$ to $5.97\times$ at $N = 40,000$. OpenMP follows a similar trend. Pthreads is the exception. Looking at the speedup when $p = 16$, it's kind of all over the place and doesn't just go up as the problem gets bigger. It's pretty clear that at 16 processors, the performance is super sensitive to how the partitions are set up and how the memory is laid out, rather than just scaling like you'd expect.

Figure 6 removes all overhead from the measurement and shows speedup on $T_{\text{computation}}$ alone. The contrast with Figure 4 is striking. MPI's computation kernel reaches $17.1\times$ speedup at $p = 16$, $N = 40,000$. This is nearly three times its overall speedup of $5.97\times$. Hybrid gets up to $11.8\times$ on computation alone. These numbers confirm that the nine-point stencil kernel itself parallelizes almost perfectly. The entire performance gap between what is theoretically achievable and what is actually observed comes from T_{other} . The implementations are well-constructed; the bottleneck is communication and synchronization overhead, not the parallel code itself.

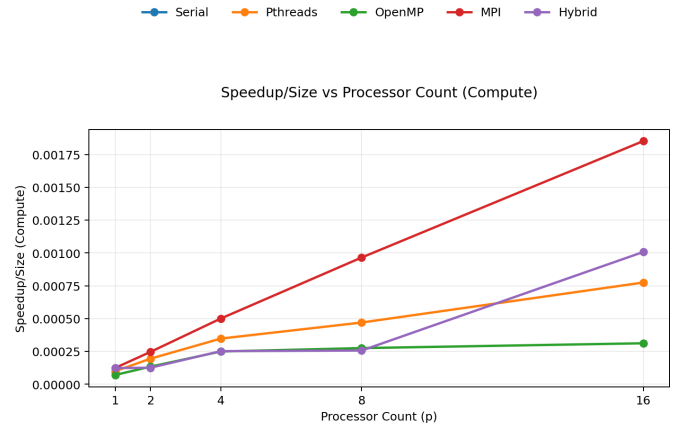


Fig. 6. Computation-only speedup vs. p , isolating the stencil kernel from all communication and synchronization overhead.

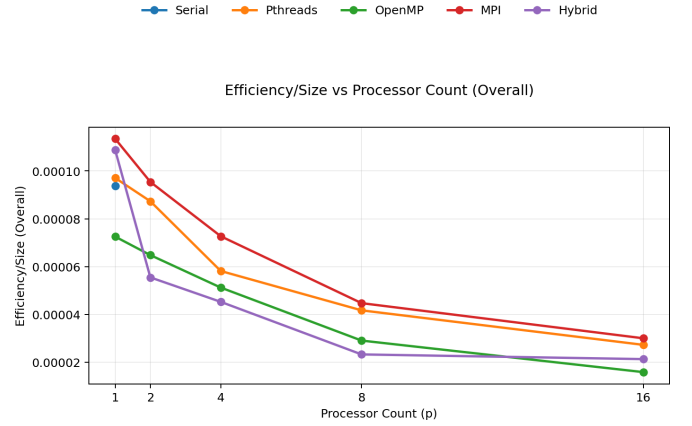


Fig. 7. Parallel efficiency vs. p for all implementations and grid sizes. A value of 1.0 represents perfect linear scaling.

D. Efficiency Analysis

Figure 7 shows parallel efficiency $E(p) = S(p)/p$ across all configurations. No implementation maintains efficiency above 50% at $p = 16$. MPI comes closest at 37.3%, followed by OpenMP at 33.0%, hybrid at 30.7%, and Pthreads at just 21.8% due to its $p = 16$ regression. The most efficient single configuration in the entire dataset is MPI at $p = 4$, $N = 40,000$, which achieves 80.0% efficiency with a $3.20\times$ speedup. This tells us that for this stencil workload, $p = 4$ is the practical sweet spot for MPI strong parallel benefit without yet being overwhelmed by communication cost. For Pthreads, the sweet spot is $p = 8$ with 62.2% efficiency, beyond which the regression described above takes hold.

Figure 8 fixes $p = 16$ and examines how efficiency changes with problem size. For MPI and OpenMP, efficiency improves as N grows, which directly supports the Gustafson's Law interpretation: larger problems spend proportionally more time in the parallel computation phase, making fixed overhead costs less significant relative to total runtime. At $N = 5,000$, MPI efficiency at $p = 16$ is 31.7%. At $N = 40,000$ it

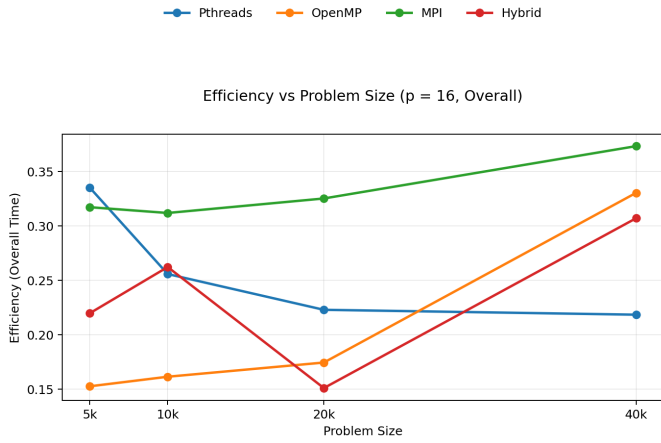


Fig. 8. Parallel efficiency vs. N at $p = 16$. Larger problem sizes generally yield better efficiency as fixed overhead costs are amortized over more computation.

risers to 37.3%. The gains are modest but consistent. Hybrid shows little improvement across sizes at $p = 16$, suggesting its overhead is not primarily driven by data volume but by the structural cost of coordinating two levels of parallelism simultaneously.

E. Overhead Analysis

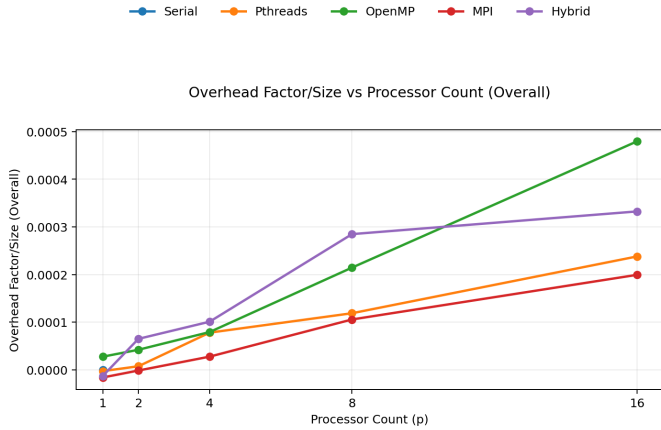


Fig. 9. Total overhead time vs. p for all implementations and grid sizes.

Figure 9 shows how total overhead grows with p . The key observation is that overhead does not grow uniformly across implementations. OpenMP’s overhead curve is nearly flat — synchronization barrier cost is essentially constant regardless of thread count, which is why OpenMP has the best weak scaling behavior of the four despite not being the fastest overall. This flat overhead profile is a structural advantage of OpenMP’s compiler-managed threading model for this type of workload.

MPI’s overhead grows more visibly with p , particularly at large N , but the growth is moderate. The dominant cost is data distribution and collection via `MPI_Scatter` and

`MPI_Gather` rather than the per-iteration ghost cell exchanges. This explains why MPI overhead at $N = 40,000$ stays between 31 and 40 seconds regardless of process count. The Scatter and Gather scale with total data volume but not rank count.

Pthreads overhead is stable across most configurations but spikes sharply at $N = 10,000$, $p = 4$, where T_{other} reaches 8.9 seconds versus roughly 1.7 seconds at every neighboring configuration. This isolated spike causes the $p = 4$ run to be slower than $p = 2$ for that grid size. The result that would be inexplicable without the overhead decomposition. It looks like a classic case of false sharing hitting right at that specific partition boundary.

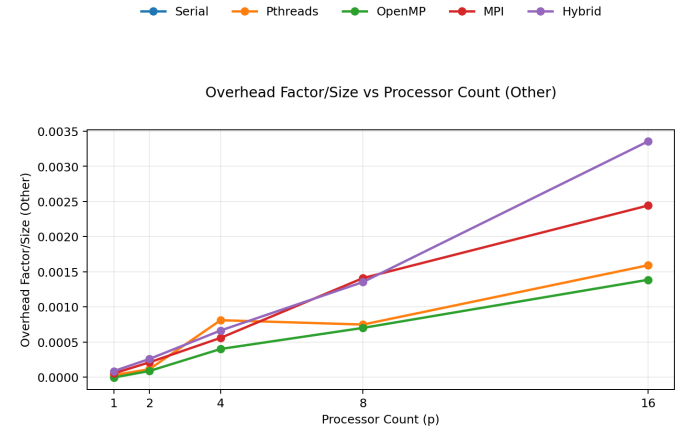


Fig. 10. T_{other} overhead vs. p for all implementations and grid sizes, isolating communication and synchronization cost.

Figure 10 isolates T_{other} specifically. The most dramatic event in the entire dataset is visible here: hybrid at $N = 20,000$, $p = 16$, where T_{other} spikes to 21.8 seconds compared to 8.6 seconds at $p = 8$ for the same grid size. A $2.5\times$ jump in absolute overhead from a single doubling of p . The cause must be architectural. Going from two MPI ranks with four OpenMP threads each to four ranks with four threads each forces data that was previously exchanged through shared memory to cross a network boundary. At this point, the hybrid overhead is actually higher than the pure MPI overhead for the same problem. It totally kills the whole idea that combining both models would be faster than just using one or the other.

At $p = 16$, overhead as a percentage of total runtime reaches 77.4% for MPI at $N = 5,000$, 68.2% for MPI at $N = 40,000$, and 79.5% for hybrid at $N = 20,000$. These numbers make clear that at $t = 13$ iterations, the fixed costs of data distribution, communication, and collection are not well amortized at high process counts. A larger value of t would push more of the runtime into the computation phase which would reduce these percentages and likely improving both speedup and efficiency across all implementations.

As shown in Figure 11, the shrinking computation bars relative to the growing overhead bars as p increases visually confirms that parallelism is effectively reducing computation time, but communication and synchronization costs are not

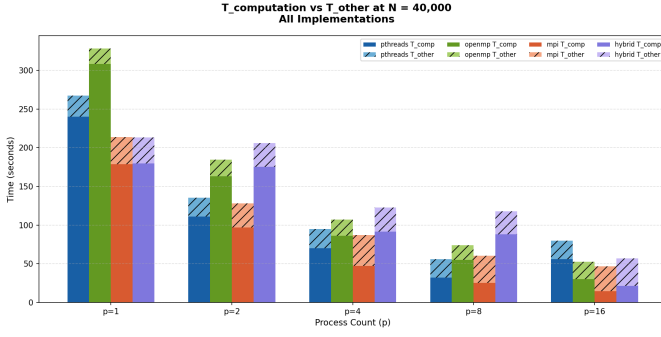


Fig. 11. Stacked bar chart of $T_{\text{computation}}$ and T_{other} for all implementations at $N = 40,000$. Solid bars represent computation time; hatched bars represent overhead. As p increases, the overhead portion grows relative to computation across all implementations, with hybrid showing the largest absolute overhead at $p = 16$.

shrinking at the same rate. At $p = 1$, nearly all runtime is computation for every implementation. By $p = 16$, overhead dominates the total bar height for MPI and hybrid in particular.

F. Workload vs. Synchronization Bottlenecks

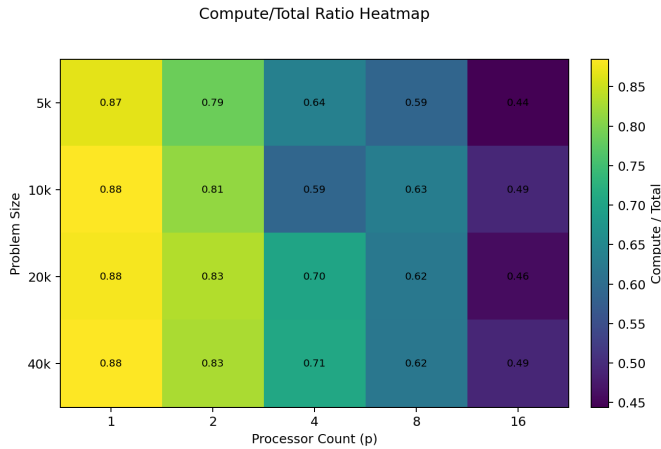


Fig. 12. Ratio of $T_{\text{computation}}$ to T_{overall} for every experiment. Lighter cells indicate a higher fraction of time spent on useful computation.

Figure 12 synthesizes all previous findings into a single view by showing what fraction of each run was spent on actual computation versus overhead. At low p , most implementations spend the majority of their time computing. As p grows, the overhead consumes an increasing share across the board.

For **Pthreads**, the ratio is healthy at moderate thread counts but degrades significantly at $p = 16$ for large grids, consistent with the barrier and cache contention effects already described. For **OpenMP**, the ratio stays relatively high even at $p = 16$, making it the most consistent implementation in terms of how productively its threads are used. For **MPI**, computation fraction drops sharply at $p = 16$ — only 31.8% of runtime at $N = 40,000$ is spent computing, meaning over two thirds of the runtime is communication and data movement. Despite this, MPI still achieves the highest overall speedup because its

computation kernel is so efficiently parallelized. For **Hybrid**, the worst case in the entire dataset appears at $N = 20,000$, $p = 16$, where only 20.5% of runtime is computation — the combined overhead of two parallel models simultaneously producing the least efficient use of resources across all 100 experiments.

Taken together, these results demonstrate that no single parallel paradigm is optimal across all configurations. MPI offers the best raw performance at large scale but pays a high communication cost. OpenMP is the most overhead-stable but slower at moderate thread counts. Pthreads delivers strong efficiency at $p \leq 8$ but becomes unpredictable beyond that. Hybrid parallelism, while theoretically appealing, does not outperform pure MPI for this workload at this iteration count, and in several configurations it is the slowest parallel option despite consuming the most resources.

V. CONCLUSION

This project evaluated the scalability of a 2D heat transfer simulation using serial, pthreads, OpenMP, MPI, and hybrid MPI+OpenMP implementations. The simulation used a 9-point stencil update and was tested across matrix sizes from 5000 to 40000 with parallel configurations from 1 to 16.

The results show that parallel computing significantly improves performance for large stencil computations. The serial implementation becomes increasingly expensive as problem size grows, while parallel versions reduce runtime substantially.

MPI achieved the best overall performance, especially for the largest problem size. This indicates that distributed-memory decomposition is highly effective for large grid-based simulations. Pthreads and OpenMP also provided strong improvements, but their scalability was limited by shared-memory effects and runtime overhead. The hybrid implementation improved over serial execution but did not consistently outperform pure MPI.

The experiment also shows that overhead must be considered carefully. Computation time often decreases significantly with added parallelism, but overhead from communication, synchronization, and I/O limits the total speedup.

Overall, this project demonstrates both the power and the limitations of parallel computing. Parallelization makes large heat transfer simulations much more practical, but performance depends heavily on the programming model, problem size, and system environment.

ACKNOWLEDGMENT

We acknowledge the use of the Expanse high-performance computing system and the course materials provided for CSCI 473.

REFERENCES

- [1] P. Pacheco and M. Malensek, *An Introduction to Parallel Programming*, 2nd ed. Morgan Kaufmann, 2022.
- [2] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large-Scale Computing Capabilities,” AFIPS Conference Proceedings, vol. 30, pp. 483–485, 1967.

- [3] OpenMP Architecture Review Board, "OpenMP Application Programming Interface," Available: <https://www.openmp.org/>
- [4] Message Passing Interface Forum, "MPI: A Message-Passing Interface Standard," Available: <https://www.mpi-forum.org/>
- [5] San Diego Supercomputer Center, "Expanse User Guide," Available: <https://www.sdsc.edu/services/hpc/expanse/>